

LOCAL NETWORK — TECHNICAL ARCHITECTURE

One function produces every euro. *Everything else is a view of it.*

How the Local Network prototype is built and why — the layering, the money calculation, the claim state machine, the determinism strategy, and where each product invariant is enforced by code rather than by policy.

Economics kernel

One door to money

Decay schedule

Quality bands

Claim state machine

Persona lens

Determinism

Invariants in code

Propose → diff → confirm

First-run empty state

Document

Implementation note — the working prototype

Audience

Chekin engineering and product

Companions

Doc 1 — Strategy & Commercial
Doc 3 — Mechanics & Dynamics

Scope

dashboard/local-network/
chekin-dashboard-preview

Contents

This document describes the working prototype, not the platform it will become. It is the companion to Doc 3 — where that document sets out what the programme should do, this one sets out how the prototype does it, and which of those decisions are load-bearing.

PART I — FOUNDATIONS

1	What the module is	2
2	Architectural principles	2
3	Layer map and dependency rules	3

PART II — THE CALCULATION

4	The economics kernel	5
5	Why fixtures store bookings, not euros	9
6	Determinism	9

PART III — BEHAVIOUR

7	The claim state machine	10
8	Personas as a lens	11
9	First-run mode	11
10	Demo session persistence	12
11	The agent layer	13
12	Rendering conventions	14

PART IV — GUARANTEES AND OPERATIONS

13	Where each invariant lives in code	15
14	Deliberate omissions and known limits	15
15	How to change things	16
16	Verification	16

Foundations

PART I

What the module does, the principles the code is organised around, and how the layers depend on one another.

1 What the module is

A **supply-side residual programme** inside the AppSell marketplace.

A host introduces a local experience provider — a *supplier* — to Chekin. Once that supplier is live, they become bookable by **every property inside their coverage area**, not just the host who introduced them. The introducing host earns an ongoing residual on every booking that supplier ever generates, anywhere in coverage:

- **scaled** by supplier quality,
- **decaying** over time to a permanent floor,
- **conditional** on the host remaining an active Chekin customer.

Three separable contributions earn: **sourced** (introduced them), **activated** (onboarded them to live), **maintained** (ongoing stewardship). Usually one host holds all three, but each can be held, released and reassigned independently.

The commercial idea only works if hosts who *didn't* source a supplier are demonstrably not worse off. That constraint drives most of the architecture below.

The non-negotiable invariants

These are not policy documents — each one is enforced somewhere specific in code, mapped in §13:

1. **Ranking is blind to sourcing.** Who sourced a supplier never affects what a guest sees.
2. **The serving host's share is never reduced to fund the residual.**
3. **The serving host always has a veto** over any network supplier or category.
4. **One level only.** No host ever earns from another host's recruitment or sourcing. There is no referral-of-hosts anywhere in this product.
5. **Suspending a claim never suspends a supplier.** Guests are insulated.
6. **Every rate is explainable on the line** — decay step, quality multiplier, bounty boost, role share.

2 Architectural principles

Money has exactly one door. `computeAccrual()` is the only function in the module that turns a percentage and a GMV into an amount. Every euro on every screen — supply book totals, a supplier's lifetime earnings, the wizard's projections, the bounty board's five-year value, the agent's answers — comes from it. Nothing else may multiply a percentage by a GMV. This

is the single most load-bearing decision in the codebase: it is what makes it *structurally impossible* for two screens to disagree about the same number.

Configuration is data, not code. Every economic constant lives in `data/config.js`. There is no hard-coded percentage, multiplier, window or cap anywhere else in the module. A `grep` for numeric literals outside that file returns only derived expressions built *from* config values. This file is the one Carlos edits after the pilot.

Explanations are computed, not written. Copy never states a rate. When a screen says a supplier earns 4.6%, that string was produced by rendering the accrual's `chain` array. If someone changes `basePoolPctOfGmv` in config, every sentence on every page updates with it, because no sentence contains a number that wasn't computed.

Invariants are enforced at the lowest possible layer. A rule that matters is not left to page authors to remember. Sorting that must ignore sourcing does so in the service. A rate that must never exceed the premium is capped in the kernel. A rejection that requires a reason throws without one.

Layers depend downward only. Pages talk to services. Services talk to the kernel and the dataset. The kernel talks to nothing. No cycles, no back-references.

3 Layer map and dependency rules

```

dashboard/local-network/
  data/
    types.js           JSDoc typedefs only – no runtime code
    config.js          window.LN_CONFIG – every economic constant
    fixtures.js        window.LN_DATA – the seeded demo dataset
  services/
    economics.js       window.LNEconomics – the calculation, pure
    network-service.js window.NetworkService – the ONLY LN_DATA consumer
    earnings-service.js money views, merged onto NetworkService
    admin-service.js   Chekin-internal views, merged onto NetworkService
  ds/
    _ln.css            module styles, extends the guest-crm vocabulary
    _ln.js             window.LN – render helpers, charts, drawer, toast
  agent/
    agent.js           window.LNAgent – scripted intents
  *.html              7 pages

```

Load order

Every page loads scripts in this order. It is a hard dependency chain, not a preference:

```

<script src="data/types.js"></script>          <!-- no-op at runtime -->
<script src="data/config.js"></script>          <!-- LN_CONFIG -->
<script src="data/fixtures.js"></script>         <!-- LN_DATA, reads LN_CONFIG -->
<script src="services/economics.js"></script>    <!-- LNEconomics, reads LN_CONFIG -->
<script src="services/network-service.js"></script>
<script src="services/earnings-service.js"></script>
<script src="services/admin-service.js"></script>
<script src="agent/agent.js"></script>           <!-- LNAgent, reads NetworkService -->
<script src="ds/_ln.js"></script>                <!-- LN, reads NetworkService -->
<script src="../../ds/_sidebar.js"></script>      <!-- shared shell -->
<script src="../../ds/_agent.js"></script>

```

Why the service layer is four files

The spec asked for one `network-service.js` and also for no file to exceed ~600 lines. A single service reached **925 lines**, so it was split along real seams rather than arbitrary line counts:

FILE	LINES	RESPONSIBILITY	READS <code>LN_DATA</code> ?
<code>economics.js</code>	199	The calculation. Premium, quality bands, decay, boosts, caps, role splits.	No — everything arrives as arguments
<code>network-service.js</code>	551	Suppliers, claims, coverage, bounties, destinations, accounts, personas, persistence	Yes — the only one
<code>earnings-service.js</code>	158	Supply book, accrual ledger, earnings summary, five-year projection	No — reads through <code>NetworkService</code>
<code>admin-service.js</code>	123	Review queue, flags, quality watch, suspensions, approve/reject	No — reads through <code>NetworkService</code>

`earnings-service.js` and `admin-service.js` **merge their methods onto `window.NetworkService`**. Pages still talk to one service object; `S.getSupplyBook()` and `S.getFlags()` work exactly as if they were defined there. This preserves two properties at once: the "single service object" ergonomics for callers, and the "single `LN_DATA` consumer" architectural rule.

`economics.js` is deliberately the only file with **no access to the dataset at all**. It cannot read `today`, so `NetworkService` injects it at boot via `E.setToday(D.today)`. That constraint is what lets the same code price a real historic booking line, a live claim, and a hypothetical supplier that does not exist yet in the submit wizard — all through one function.

The calculation

PART II

The money. One function produces every euro in the module — this part covers what it does, why the caps sit where they do, and why the fixtures deliberately hold no currency at all.

4 The economics kernel

4.1 The base platform, for context

Shown in the UI purely so hosts can verify nothing was taken from them. These mirror the AppSell marketplace doc §17 and are not editable in this module:

CONSTANT	VALUE	MEANING
<code>baseNrpPctOfGmv</code>	20%	Standard commission on external supply
<code>servingPropertySharePctOfExternalNrp</code>	40%	What the property serving the guest keeps → 8% of GMV
<code>pmsPartnerSharePctOfNrp</code>	30%	What the PMS/channel partner keeps → 6% of GMV

4.2 Where the residual money comes from

A locally-sourced supplier who is **not** already on an aggregator accepts a higher commission — roughly 25% instead of 20% — because Chekin brings them demand they had no other route to. Those extra **5 points**, and only those, fund the sourcing pool.

```
networkPremiumPctOfGmv: 5    // the extra the supplier accepts
basePoolPctOfGmv:         4    // pool before quality, decay and boosts
```

The serving host's 8% and the PMS partner's 6% are computed off the base 20% and are **arithmetically untouched** by any of this. That is invariant #2, and [discover.html](#) renders both splits side by side so a host can see it rather than be told it.

4.3 Saturation — not paying for supply we already have

`config.js` supplies `saturatedPremiumPctOfGmv: 1` but not the predicate for *when* a category counts as saturated. The rule implemented is:

```
function premiumFor(destination, supplier) {
  var saturated = destination.supplyDensity === 'high' &&
    destination.categoriesCovered.indexOf(supplier.category) >= 0;
  return saturated ? C.saturatedPremiumPctOfGmv : C.networkPremiumPctOfGmv;
}
```

Data-driven, so no magic numbers, and visible in the fixtures: Barcelona is `supplyDensity: 'high'` and covers `dining`, `transfer`, `storage`, `wellness`, `tickets`, `mobility` — so a Barcelona dining supplier is capped at a **1%** premium, while the identical business in Cádiz gets the full **5%**. The contrast is the whole point of seeding two destinations.

4.4 Quality bands

0.90–1.00	×1.25	Excellent
0.75–0.89	×1.00	Good
0.60–0.74	×0.60	At risk
0.00–0.59	×0.00	Failing

Nuance: the published bands have gaps — a score of 0.895 falls between `0.89` and `0.90`. Rather than let that resolve to nothing, bands are matched **highest-first on `score >= min`**:

```
for (var i = 0; i < C.qualityBands.length; i++) {
  if (score >= C.qualityBands[i].min) return C.qualityBands[i];
}
```

So 0.895 resolves to *Good*. The band edges remain exactly as published; only the matching is made total.

Below `qualityFloorForAccrual` (0.60) nothing accrues at all — which the `×0.00` multiplier would produce anyway, but it is also checked explicitly so the UI can give a *reason* rather than silently showing zero.

4.5 Decay

Months counted from the claim's go-live date, where **month 1 is the first month live**:

MONTHS	MULTIPLIER	LABEL
1–12	×1.00	Year 1
13–24	×0.80	Year 2
25–36	×0.60	Year 3
37+	×0.40	Year 4+ (floor)

The residual steps down and then **holds forever** while the claim is live and the host is an active customer. It never reaches zero. `sup_chef` in the fixtures has been live 40 months precisely so the floor is visible in the supply book without time travel.

Decay is evaluated **as of a date**, not as of today:

```
function computeAccrual(gmv, claim, supplier, destination, asOf)
```

Historic order lines pass their own date, so a booking from 18 months ago is priced at the decay step that applied *then*. This is what makes the monthly earnings bar chart show real step-downs rather than a flat rate retroactively applied.

4.6 The full calculation, in order

```
poolPct = min(basePoolPctOfGmv × qualityMultiplier(score), premium) // cap #1
poolPct *= decayMultiplier(monthsSince(claim.liveAt))
poolPct *= claim.bountyId && within bounty window ? 1.50 : 1
poolPct *= sharingMode === 'delayed' && within boost window ? 1.15 : 1
poolPct = min(poolPct, premium) // cap #2
poolAmount = gmv × poolPct / 100
// then split across the roles actually held
```

Why two caps, and why the order matters. Cap #1 runs *before* decay. That means an excellent supplier's $\times 1.25$ uplift is spent getting to the premium ceiling, and decay then applies to the capped figure rather than to a number that was never payable. Cap #2 catches the boosts. Removing either cap, or reordering them, changes the answer — this ordering is specified, deliberate, and implemented verbatim.

4.7 Worked examples, verified against the fixtures

Each is a real claim in the seeded dataset, and these strings are what the UI actually renders:

CASE	CHAIN	RESULT
Year-1 excellent, saturated category (<code>sup_gastro</code> , BCN dining)	4.0% base $\times 1.25$ excellent $\rightarrow$ capped at 1.0% $\times 1.00$ decay	1.0%
At-risk supplier on watch (<code>sup_horse</code> , Cádiz)	4.0% base $\times 0.60$ at risk $\times 1.00$ decay	2.4%
Bounty-sourced, boost active (<code>sup_kayak</code>)	4.0% base $\times 1.00$ good $\times 1.00$ decay $\times 1.50$ bounty $\rightarrow$ capped at 5.0%	5.0%
Year-4 floor (<code>sup_chef</code> , live 40 months)	4.0% base $\times 1.00$ good $\times 0.40$ decay $\cdot$ year 4+	1.6%
Delayed sharing, boost active (<code>sup_chiringuito</code>)	4.0% base $\times 1.00$ good $\times 1.00$ decay $\times 1.15$ delayed-sharing	4.6%
Saturated + year 2 (<code>sup_transfer</code> , BCN)	4.0% base $\times 1.00$ good $\rightarrow$ capped at 1.0% $\times 0.80$ decay $\cdot$ year 2	0.8%
In delayed exclusivity (<code>sup_ceramics</code>)	"In delayed exclusivity — the residual starts when the window closes."	0%

A NOTE ON THE SPEC'S ILLUSTRATIVE EXAMPLE

The brief shows `4.0% base  $\times 1.25$  quality  $\times 0.80$  decay  $\times 1.50$  bounty  $\rightarrow$  capped at 5.0%  $\rightarrow$  €4.50`. That exact combination is **not reachable** with the configured windows: the bounty boost runs 12 months, but $\times 0.80$ decay does not begin until month 13. The renderer produces precisely that format — verified against a year-1 excellent bounty claim, which caps at 5.0% and pays €4.50 on €90 of GMV — but that specific set of factors cannot co-occur. The economics were implemented exactly as specified; the example is treated as illustrating the *format*, not a reachable state. **This is worth confirming when the schedule is revisited after the pilot.**

4.8 Sharing modes

MODE	DURING	RESIDUAL
private	Always	None. Serves only the sourcing host's properties — you are buying exclusivity, not a residual
network	Always	Full rate from day one
delayed	Exclusivity window (60 or 90 days)	None — the exclusivity is the compensation
delayed	After the window, for 12 months	×1.15 boost
delayed	Thereafter	Normal rate

Interpretation flagged: the brief specifies `delayedBoostMultiplier` and `delayedBoostMonths` but not what happens during exclusivity. The reading implemented is that a supplier serving only your properties generates no network pool, because there is no network booking to take a premium from. The claim is modelled with `networkAt: null` until the window closes, and `blockedReason()` returns a human-readable explanation rather than a bare zero. `sup_ceramics` demonstrates the countdown state; `sup_chiringuito` demonstrates the active boost afterwards.

4.9 Role splits and the rounding rule

sourced 30% · activated 45% · maintained 25% (must sum to 100)

Only roles **actually held** are paid. A released role's share is **not** redistributed to the other holders — it stays with Chekin. This is invariant #4 expressed in arithmetic: there is no mechanism by which one account's share can grow because of another account's work. `clm_kayakbcn` in the fixtures holds only `sourced` + `maintained` (55%), because Chekin ops ran the onboarding.

Naive rounding here produces a real bug: `round2(x × 0.30) + round2(x × 0.45) + round2(x × 0.25)` can exceed `round2(x)` by up to 1.5 cents. The parts summing to more than the whole is the kind of thing that surfaces as a broken total on a totals row. The fix computes the target first and absorbs the drift into the largest share:

```
var allocated = round2(amount * heldPct / 100);
var diff = round2(allocated - sum(splits));
if (diff !== 0) splits[largestIndex].amount = round2(splits[largestIndex].amount + diff);
```

4.10 Blocked reasons

Rather than returning `0` and leaving pages to guess why, `blockedReason(claim, supplier)` returns a sentence:

- private mode → "Private mode — this supplier serves only your properties, so no network pool accrues."
- delayed, pre-network → "In delayed exclusivity — the residual starts when the exclusivity window closes."
- suspended → "Claim suspended — accrual is zero until it is reinstated. The supplier stays bookable."
- dormant → "Claim dormant — the sourcing host is no longer an active Chekin customer."
- below the quality floor → "Score is below the quality floor of 0.60 — nothing accrues."

The accrual is still returned as a complete object with `poolAmount: 0` and a two-element `chain`, so the breakdown renderer never needs a null branch.

5 Why fixtures store bookings, not euros

`data/fixtures.js` contains **no percentages and no residual amounts**. It stores 663 raw order lines:

```
{ id, claimId, supplierId, servingPropertyId, gmv, date }
```

Every euro shown anywhere is derived at read time by running each line through `computeAccrual` with that line's own date. The consequences are worth stating explicitly:

- **Changing `config.js` immediately changes all historic earnings.** Halve `basePoolPctOfGmv` and every chart, total and lifetime figure in the module halves, consistently, with no fixture edits. This is exactly what you want when tuning economics after a pilot.
- **Fixture data cannot contradict the calculation**, because it never expresses an opinion about money.
- Trend sparklines, month-over-month comparisons and the decay curve are all genuinely derived, so they show real step-downs at the right months.

The cost is recomputation on every read. At this dataset size (663 lines × a handful of views) it is imperceptible, and a real implementation would post accruals server-side anyway. The signature is already shaped for that: `computeAccrual` is pure and takes everything it needs as arguments.

6 Determinism

A demo that shifts under you is worse than no demo. Three rules keep it stable:

A fixed `today`. `LN_DATA.today = '2026-06-13'`, and `Date.now()` / `new Date()` are never called at module scope. Every relative date, countdown and decay step is computed against that anchor. Without this, fixtures drift daily and the countdown chips break.

No `Math.random()`. Fixture variation comes from an FNV-1a hash of a seed string:

```
function h(s) { var x = 2166136261; for (...) { x ^= s.charCodeAt(i); x = (x * 16777619) >>> 0; }
return x >>> 0; }
function rnd(s) { return (h(s) % 10000) / 10000; }
```

Seeds are composed from stable identifiers (`claimId + 'm' + month + 'l' + index`), so the same booking always has the same value on every reload, on every machine.

The stored snapshot must agree with the computed value. `Claim.decayStep` is stored on the record *and* recomputed by `decayFor()`. A test asserts they never disagree, which catches fixture edits that move a `liveAt` across a decay boundary without updating the snapshot.

Behaviour

PART III

How the module behaves over time and across the people using it: the claim lifecycle, the persona lens, the empty state, and the agent's contract.

7 The claim state machine

```
draft → screening → pending_supplier → onboarding → live ⇄ watch → suspended → terminated
                        ↓ 30d                ↓ 60d                ↓
                        expired             role released          dormant → released
```

Transitions are declared, not implied, and `advanceClaim()` refuses anything not on the list:

```
var TRANSITIONS = {
  draft:      ['screening'],
  screening:  ['pending_supplier', 'terminated'],
  pending_supplier: ['onboarding', 'expired'],
  onboarding: ['live', 'terminated'],
  live:       ['watch', 'suspended', 'dormant', 'terminated'],
  watch:      ['live', 'suspended', 'terminated'],
  suspended:  ['live', 'watch', 'terminated'],
  dormant:    ['live', 'released'],
  expired: [], released: [], terminated: [] // terminal
};
```

An illegal transition throws with a readable message (`"Cannot move a claim from draft to live."`) rather than silently corrupting state.

Timed states and countdowns

STATE	WINDOW	CONFIG KEY	ON EXPIRY
<code>pending_supplier</code>	30 days	<code>supplierConfirmDays</code>	Claim expires; supplier returns to the open pool
<code>onboarding</code>	60 days	<code>activationDays</code>	The <i>activated</i> role is released; sourced is retained
<code>dormant</code>	12 months	<code>dormantReactivationMonths</code>	Claim released; supplier becomes unclaimed
delayed exclusivity	60 or 90 days	<code>delayedExclusivityDays</code>	Opens to network with the ×1.15 boost

`countdownFor(claim)` derives days remaining from the relevant **timeline event** rather than from `createdAt` — the confirmation clock starts when the invitation was sent, not when the draft was created. `LN.countdownChip()` colours it: blue normally, amber at ≤10 days, red when overdue.

The separation that matters

Claim state and supplier status are different fields with different meanings. Suspending a claim sets `claim.state = 'suspended'` and zeroes the residual; the supplier stays `live` and fully bookable. Suspending a *supplier* is a guest-safety action that stops bookings. `admin-review.html` presents these as two distinct buttons with copy explaining the difference, because conflating them would let a commercial dispute about who earns degrade the guest experience. That is invariant #5.

All eleven states are present in the seeded fixtures, including a claim expired because the supplier never confirmed (`clm_museo`), one dormant from a churned host (`clm_celler`), one voided by two suspensions in 12 months (`clm_quad`), and one fully released after the reactivation window closed.

8 Personas as a lens

Five personas — `host`, `pm`, `supplier`, `chekin`, `admin` — persisted in `localStorage['ln_persona']`, default **PM**, and deep-linkable via `?persona=pm` (mirroring `?first_run=1`).

These are not five apps. Each page has one render path that branches on persona and on the *account* the persona maps to:

```
personaAccounts: { host: 'acc_host_marta', pm: 'acc_pm_gaditana',
                    supplier: 'acc_sup_riera', chekin: null, admin: null }
```

Most differentiation falls out of the data rather than from branching. `getSupplyBook(accountId)` returns a different book per account with no persona-specific code. `supplier-detail.html` shows a host their own earnings and shows another host's claim as *"held by Costa Habitat, so these figures are theirs, not yours"* — same code path, different account.

Where personas genuinely differ, they differ structurally:

- **Chekin / admin** have `null` accounts. They see programme-wide views (every claim across all accounts) and internal tools. `supply-book.html` flattens one book per account.
- **Supplier** is read-mostly: bookings, score, profile completeness, and a *Confirm the introduction* action when the claim is `pending_supplier`.
- **Submit is guarded** for `supplier`, `chekin` and `admin` — a claim is always held by a host account. Internal sourcing goes to the central catalogue and creates no claim, so the wizard explains that rather than crashing on a null account. The subnav hides the tab for those personas, and the page guards the direct link independently.

`LN.ROUTES` carries an optional `personas` array; `subnav()` filters on it.

9 First-run mode

This product **launches empty**, so empty states matter more than populated ones. `?first_run=1` sets `localStorage['ln_first_run']` and the service serves a reduced dataset.

The question is what genuinely pre-exists on day one:

KEPT	EMPTIED
Destinations	All claims
Accounts and their properties	All order lines / earnings
Centrally-sourced suppliers (<code>origin === 'central'</code>)	All network-sourced suppliers
Bounties	Activity feed

Bounties survive deliberately: Chekin publishes them to *bootstrap* a destination, so they exist before any host has done anything. Central suppliers survive because the AppSell marketplace predates this programme.

The first-run home is a genuine sell rather than a blank page — and its illustrative earnings example is **computed from LN_CONFIG at render time**, not written:

```
var y1 = S.computeAccrual(gmv, demoClaim, demoSup, dest); // year 1
for (var y = 0; y < 5; y++) { ... }                      // 5-year total
```

Change the decay schedule and the first-run pitch updates itself. A hard-coded example that drifts from the real economics is a trap, particularly on the one screen whose job is to be persuasive.

Verified across all seven pages: every one shows a real empty state, none shows a broken populated one.

10 Demo session persistence

This is a deliberate exception to "no persistence beyond localStorage for demo flags," and it is worth understanding why.

Acceptance required that the submit wizard "produces a claim visible in the supply book and the admin queue." Fixtures are in-memory, and the wizard redirects to `supplier-detail.html` — a full page load, which re-evaluates `fixtures.js` and discards everything. Without persistence, that criterion is unmeetable.

The implementation snapshots only the mutable slices to `localStorage['ln_snapshot']` and restores them before any read:

```
{ v: 1, suppliers, claims, bounties, properties, accounts, activity }
```

Order lines are never written — earnings stay derived. A newly created claim therefore correctly shows **zero earnings and a blocked reason**, because a claim in screening has no bookings.

Two details keep it honest:

- **Every write goes through one door.** Rather than calling `snapshot()` from fifteen places and eventually forgetting one, the mutating methods are wrapped at export time:

```
[ 'createClaim', 'advanceClaim', 'setSharingMode', ... ].forEach(function (k) {
  var orig = NetworkService[k];
  NetworkService[k] = function () { var r = orig.apply(null, arguments); snapshot(); return r; };
});
```

A method that throws never snapshots, so a rejected write leaves no trace.

- **SNAP_VERSION** invalidates stale snapshots when the fixture shape changes — otherwise an old snapshot would silently override new fixtures and look like a data bug.

A **Reset demo** button appears in the persona bar whenever a snapshot exists. First-run mode never snapshots.

11 The agent layer

`window.LNAgent` follows the guest-crm `CRMAgent` pattern: **every intent returns a structured object, never a string**, with a `reasoningSummary` so the UI can always show the "why". Responses are rule-based against the fixtures — no LLM — so the demo is stable.

Critically, the agent **shares the service layer with the pages**. It has no numbers of its own. When it says the book is worth €3,495 over five years, that came from `S.projectResidual()` — the same call the supply book makes.

Read intents

INTENT	RETURNS
"What should I source in Cádiz?"	Ranked open bounties with year-1 and five-year projected residual
"Why did my residual drop last month?"	Per-claim deltas, each attributed to the factor that actually moved
"Which of my suppliers are at risk?"	Watch-list with the falling score driver
"How much is my supply book worth?"	Run-rate, five-year curve, top claims

Attribution is real, not narrated. `whyResidualChanged()` recomputes both months through `computeAccrual` and compares factor by factor in priority order — decay step, then quality band, then bounty boost, then delayed boost, then pool rate, then volume, then value:

```
Bodega Riera  -10.75  the rate held; booking volume changed (4 → 3 bookings)
Kayak Caleta   +8.15   the rate held; booking value changed (€273 → €436 across 2 bookings)
```

The last two branches matter: an early version reported "volume changed (2 → 2 bookings)" for a value-only move, which is the kind of confidently wrong explanation that destroys trust in an assistant.

Mutating intents — propose → diff → confirm

Nothing mutates on `ask()`. A mutating intent returns a **proposal** with an explicit diff and `requiresConfirm: true`. Only `LNAgent.apply(proposal)` writes.

Q: Block the kayak operator for my Gràcia properties
 A: Block Kayak Costa Brava Express on 1 of your 16 properties.
 Gràcia Loft 3B: Bookable → Blocked

Two matching nuances

The property hint names a place, not a supplier. "for my Gràcia properties" contains *Gràcia*, which also matches *Aigües de Gràcia Spa* — and did, in an early version, blocking the wrong supplier. The hint clause is now stripped before supplier matching, and the destination it resolves to is used as a **tie-breaker**: "the kayak operator for my Gràcia properties" correctly picks the Barcelona kayak operator over the Cádiz one.

A no-op is reported as a no-op. `proposeMode()` on a claim already in that mode returns `type: 'no_change'` with the current rate, instead of a diff full of `network → network`.

12 Rendering conventions

One money helper. `LN.money()` is the only place a currency string is formatted. Cents appear on line-level amounts and are suppressed on rounded totals.

A percentage never appears without its chain. `LN.chain(accrual)` renders the `chain` array as segments joined by `×` and `→` operators, colour-coded by kind. There is no helper that prints a bare rate, which is invariant #6 enforced by omission.

Status colour semantics, applied without exception:

STATES	COLOUR
<code>live</code>	green
<code>screening</code> , <code>pending_supplier</code> , <code>onboarding</code>	blue
<code>watch</code>	amber
<code>suspended</code> , <code>expired</code> , <code>terminated</code>	red
<code>draft</code> , <code>dormant</code> , <code>released</code>	grey

One CSS block (`.st-*`) and one renderer (`LN.statePill`) drive every pill in the module.

Charts are hand-rolled inline SVG — no libraries. `LN.sparkline()`, `LN.decayCurve()` (with decay-step markers and rate labels), `LN.barChart()` (monthly residual with `<title>` tooltips), and `LN.coverageMap()` (a CSS-gradient grid with a scaling coverage circle).

Tokens only. Every colour, font, radius and shadow comes from `../ds/colors_and_type.css`. `_ln.css` extends the guest-crm component vocabulary (`.card`, `.kpi`, `.badge`, `.chip`, `.empty`, `.grid3`, `.drawer`, `.toast`, `.stepper`) and adds module primitives: `.chainrow`, `.claimsteps`, `.gauge`, `.splitbar`, `.covmap`, `.mode`, `.ln-invariant`. Nothing forks.

Guarantees and operations

PART IV

Where each product invariant is actually enforced, what was deliberately left out, how to change the economics, and what the test suites cover.

13 Where each invariant lives in code

#	INVARIANT	ENFORCED AT
1	Ranking is blind to sourcing	<code>getSuppliers()</code> sorts by <code>score</code> then <code>name</code> — <code>origin</code> and <code>sourcingAccountId</code> are never sort keys. <code>discover.html</code> re-sorts by score then distance. Sourcing appears only as an <code>.attrib</code> label after the fact, with a comment marking it non-ordering
2	The serving host's share is never reduced	The pool is drawn from <code>networkPremiumPctOfGmv</code> and capped there twice. The serving share is computed off <code>baseNrpPctOfGmv</code> and is never an input to <code>computeAccrual</code> . <code>LN.splitBar()</code> renders both cases; <code>discover.html</code> states it in a persistent banner and proves it in a drawer
3	The serving host has a veto	<code>blockSupplierForProperty</code> , <code>blockSupplierForAccount</code> , <code>blockCategoryForAccount</code> . Exposed on <code>discover.html</code> (per supplier and per category), on the home "New in your coverage" feed, and through the agent
4	One level only	There is no data path from one account to another's earnings. <code>roleSplits</code> pays only current holders; a released share goes to Chekin, never to another host. No referral, downline, tier or recruitment concept exists in the schema — a language scan across the module returns clean
5	Suspending a claim never suspends a supplier	<code>Claim.state</code> and <code>Supplier.status</code> are separate fields. <code>advanceClaim(id, 'suspended')</code> touches only the claim; <code>suspendSupplier()</code> is a separate method with separate UI and explanatory copy
6	Every rate is explainable	<code>computeAccrual</code> builds a <code>chain</code> array on every path, including the zero paths. <code>LN.chain()</code> is the only rate renderer

14 Deliberate omissions and known limits

Not built, on purpose: authentication, a backend, real API calls, real persistence, mobile layouts (tablet-down is supported), and the P1 pages (`earnings.html` line-level ledger with CSV export, `supplier-portal.html`).

The shared agent panel is mounted but not wired. `../ds/_agent.js` is monolithic — its panel content and canned replies are hard-coded, with no extension hook. Wiring `LNAgent` into it would require editing `ds/`, which the brief restricts to the sidebar entry. `LNAgent` is complete and tested; connecting it is a small change to `ds/_agent.js` whenever that file is opened for extension.

Coverage geometry is illustrative. `propertiesInCoverage` scales linearly on `km / capKm`. Real coverage is area-based and would come from an isochrone service. The number is honest about being a demo figure; the `cap` per category is real and enforced.

`getPropertiesInCoverage()` uses a **modulo heuristic** to pick which properties fall inside a radius, rather than true geometry. `getNetworkSupplyForProperty()` does compute real haversine distances.

Fixture GMV is small-supplier scale — roughly €700–15,000 per supplier per year, which puts the PM persona's book at about €1,030/year run-rate and €3,495 over five years. `assumedAnnualGmvByCategory` in config was calibrated against the observed book so the wizard's projections and real earnings tell the same story. If the real market is larger, both move together.

15 How to change things

Change the economics. Edit `data/config.js` and nothing else. Every rate, projection, chart, chain and explanatory sentence in the module updates. Run `node check.js` (see §16) to confirm the invariants still hold.

Add a category. Add it to `coverageCapsKm` in config, add a label in `LN.CAT` in `ds/_ln.js`, and optionally add an entry to `assumedAnnualGmvByCategory`. Add it to a destination's `categoriesCovered` if it should be saturation-capped there.

Add a destination. Append to `destinations` in `fixtures.js` with `supplyDensity`, `activeProperties`, `liveSuppliers` and `categoriesCovered`. Saturation, premiums and bounty projections follow automatically.

Change the decay schedule. Edit `decaySchedule`. Also update `stepFor()` in `fixtures.js` if the boundaries move, since it snapshots `Claim.decayStep`, and a test asserts the two agree.

Wire a real API. Every service method is synchronous but shaped to be wrapped: reads take a filter object and return a plain value; writes take an id and a payload. Replace the bodies in `network-service.js` with `fetch` calls and return Promises. `economics.js` stays untouched — it is already pure, dataset-free, and the natural thing to port server-side so client and server cannot disagree about money.

16 Verification

Three suites, all currently passing.

1 • Browser pass — 42 combinations, 0 failures. Every page × every persona × first-run, driven through headless Chrome with console capture. Fails on any console error or on a suspiciously small DOM, so a page that silently renders nothing is caught as well as one that throws. The error detection was itself verified against a deliberately broken page before being trusted.

7 pages × 5 personas + 7 pages × first-run = 42 checks

2 • Economics harness — 55 assertions. Hand-computed expectations for every worked example in §4.7, plus: all eleven claim states present; all four decay steps represented; stored `decayStep` agrees with `decayFor()`; role splits never exceed the pool; boosts cap at the premium; illegal transitions throw; rejection without a reason throws.

3 • End-to-end and agent — 27 assertions. The wizard flow across a simulated page reload (fresh globals, re-evaluated scripts, shared `localStorage`): create a claim → it appears in the supply book and admin queue with zero earnings and a blocked reason → an admin advances it through `pending_supplier` and `onboarding` to `live` → it starts earning with the bounty boost on the chain → reset restores the seeded fixtures. Plus all six agent intents, and a check that a proposal alone mutates nothing until `apply()` is called.

Static checks. No file exceeds ~600 lines (largest: `network-service.js` at 551). No economic literal outside `config.js` .
No recruitment, downline, tier-of-people or MLM language anywhere in the module.